

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from scipy.stats import f_oneway

# Load and clean the data

file_path = "baseball_ratios.csv"
df = pd.read_csv(file_path)

# Drop categorical columns not used in clustering
df = df.drop(columns=['year', 'team_name'])
df = df.dropna()

print(df.head())

# Normalize the data

scaler = StandardScaler()
normalized_data = scaler.fit_transform(df)
normalized_df = pd.DataFrame(normalized_data, columns=df.columns)

head_normalized = normalized_df.head()
summary_normalized = normalized_df.describe()
print(head_normalized)
print(summary_normalized)

# Elbow method to choose number of clusters

cluster_range = range(1, 11)
inertias = []
for k in cluster_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10).fit(normalized_data)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(cluster_range, inertias, marker='o', linestyle='--')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')

```

```
plt.ylabel('Inertia')
plt.grid(True)
plt.show()
```

```
for k, inertia in zip(cluster_range, inertias):
    print(f"Number of clusters: {k}, Inertia: {inertia:.2f}")
```

```
# PCA for visualization and KMeans on the normalized data
```

```
# Use the SCALED data for PCA, not the raw dataframe, since PCA is
# sensitive to feature scale (e.g. games_played vs fielding_percentage_pg).
X = normalized_data
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print(pca.explained_variance_ratio_)
```

```
k_final = 3 # chosen from the elbow plot above
kmeans = KMeans(n_clusters=k_final, random_state=42, n_init=10)
kmeans.fit(X_pca)
y_kmeans = kmeans.predict(X_pca)
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=50)
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X')
plt.title('KMeans Clusters after PCA Reduction')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```

```
# Cluster summary statistics
df['Cluster'] = kmeans.labels_
unique_clusters = df['Cluster'].unique()
```

```
cluster_means = df.groupby('Cluster').mean()
cluster_medians = df.groupby('Cluster').median()
cluster_stds = df.groupby('Cluster').std()
cluster_counts = df.groupby('Cluster').size()
```

```
print("Cluster sizes:")
print(cluster_counts)
print("\nCluster means:")
```

```

print(cluster_means)

# Top 10 features with the biggest differences between cluster means
feature_variance = cluster_means.var().sort_values(ascending=False)
most_distinctive_features = feature_variance.index[:10]
print("\nTop features (highest variance between clusters):")
print(feature_variance.head(10))

# ANOVA to find statistically significant differences between clusters
anova_results = {}
feature_columns = [col for col in df.columns if col != 'Cluster']
for feature in feature_columns:
    feature_by_cluster = [
        df[df['Cluster'] == c][feature].dropna() for c in unique_clusters
    ]
    # Only run ANOVA if we have sufficient data in each group
    if all(len(group) > 1 for group in feature_by_cluster):
        anova_result = f_oneway(*feature_by_cluster)
        anova_results[feature] = {
            'F-statistic': anova_result.statistic,
            'p-value': anova_result.pvalue
        }

anova_df = pd.DataFrame(anova_results).T
anova_df = anova_df.sort_values('F-statistic', ascending=False)
print("\nANOVA Results (most significant differences):")
print(anova_df.head(10))

#export results
df['PCA1'] = X_pca[:, 0]
df['PCA2'] = X_pca[:, 1]

output_file_path = "baseball_clusters.csv"
df.to_csv(output_file_path, index=False)
print(f"Data exported to {output_file_path}")

```